



Tecnológico Nacional de México

Instituto Tecnológico Superior de Felipe Carrillo Puerto

Ingeniería en Sistemas Computacionales



Asignatura

Programación BackEnd

Tema II

API's

EVIDENCIA DE APRENDIZAJE

AA 2.2 Axios y autenticación

Profesor

MTI. Paloma Gongora Sabido

Alumno (s):

Adan Adair Moo Noh

ISC-8B

Felipe Carrillo Puerto a 13 de abril de 2026





API REST

Objetivo: Comprender los diferentes tipos de autenticación en API REST y realizar pruebas con ejemplos sencillos utilizando Postman y Axios.

Sección 1: ¿Qué es Axios?

AXIOS

Promise based HTTP client for the browser and node.js

[Website](#) • [Documentation](#)

npm v1.8.4 cdnjs v1.8.4 CI passing Gitpod Ready-to-Code coverage 94% install size 2.62 MB
minzipped size 13.6 KiB downloads 251M/month chat on gitter code helpers 150 Snyk security monitored

[Axios](#) es una biblioteca de JavaScript utilizada para realizar solicitudes HTTP desde el navegador y Node.js.

Simplifica el proceso de hacer solicitudes GET, POST, PUT y DELETE a APIs, facilitando la interacción con servicios web.

Al usar promesas, puedes manejar respuestas de manera efectiva, utilizando `.then()` para respuestas exitosas y `.catch()` para errores.

Puede agregar fácilmente cabeceras de autenticación, como tokens o claves API, a tus solicitudes.

La sintaxis simple y clara permite concentrarse en la lógica de la aplicación en lugar de preocuparse por los detalles de las solicitudes HTTP.

Sección 1 Uso de API Fetch para hacer una solicitud GET

1. Crea carpeta AA 2.2 API
2. Crea archivo index.js
3. Inicializa el proyecto
4. Cambia a tipo Modulo
5. Transcribe en el index lo siguiente:



```
index.js > ...
1  fetch('https://jsonplaceholder.typicode.com/posts')
2    .then (respuesta => {
3
4      if (!respuesta.ok) {
5        throw new Error('Respuesta del servidor fallida ' + respuesta.statusText);
6      }
7
8      return respuesta.json();
9    })
10
11   .then (datos => {
12     console.log ('Datos recibidos: ', datos);
13   })
14
15   .catch(error => {
16     console.error('Error al hacer la solicitud:', error);
17   });
```

6. Ejecuta y observa los resultados. Para comprender cómo API Fetch accede a las API's consulta:
 - a. [API Fetch JS de freeCodeCamp](#)
 - b. [Uso de Fetch de MDN](#)
7. Crea otro archivo nuevo app.js
8. Transcribe el siguiente código:

```
app.js > ...
1  import axios from 'axios';
2
3  axios.get('https://jsonplaceholder.typicode.com/posts')
4    .then(respuesta => {
5      console.log('Datos recibidos:', respuesta.data);
6    })
7
8    .catch(error => {
9      console.error('Error al hacer la solicitud:', error);
10    });
```

9. Contesta con tus propias palabras lo siguiente:



- a. ¿Cómo se manejan los errores en Axios en comparación con Fetch, y qué información adicional proporciona Axios sobre los errores?

R= En axios el manejo es más automático, ósea cualquier código http que está fuera de rango de éxito como el “404 o 500” activa se bloquee, mientras que en fetch solo entra al bloque `.catch()` y si hay un error de red o la solicitud no se pudo completar. Si el servidor responde con un error como un “404”.

- b. ¿Axios permite interceptar solicitudes y respuestas antes de que sean manejadas? ¿Cómo se realiza esto en comparación con Fetch?

R= En cuestion de axios si permite interceptar solicitudes y respuestas de manera nativa gracias a `axios.interceptors`. Esto nos sirve para añadir automáticamente tokens de autenticación a cada envío o para transformar los datos, mientras que en fetch no tiene esa función aunque para lograr algo similar se debería “envolver” la función fetch global en una función personalizada que añada la lógica necesaria.

- c. ¿En qué sección del código de tu archivo `app.js` se convierten los datos a formato JSON cuando usas Axios?

R= En axios la conversión ocurre de forma automática detrás de escena, los datos ya están listos para usarse dentro de `respuesta.data`. mientras que en fetch la conversión es manual, usando el método `.json()`.

- d. ¿Cuál es el impacto en el tamaño del directorio `node_modules` al instalar Axios en comparación con el uso de la API Fetch, que es nativa de JavaScript y no requiere instalación adicional?

R= En axios al ser una biblioteca externa, al instalar axios aumenta el tamaño de la carpeta `node_modules` y el peso final de la aplicación suele ser aproximadamente de 2.62 mb, mientras que en fetch el impacto es nulo (0 bytes) en `node_modules` ya que es una API nativa.



Programando con frameworks de JavaScript

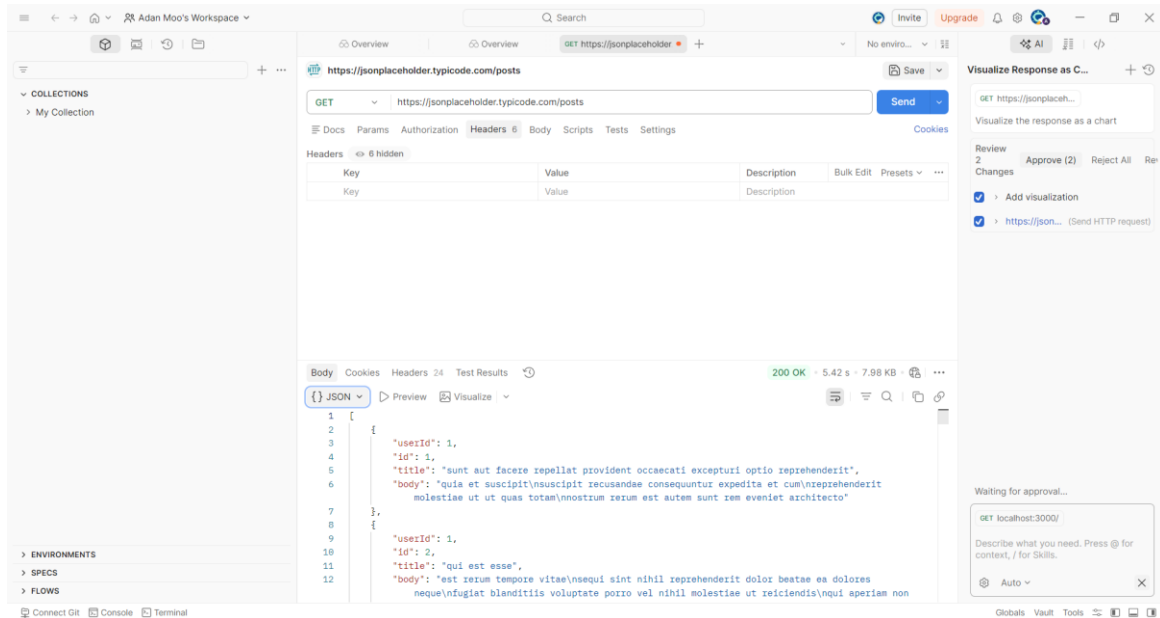


2026
año de
**Margarita
Maza**



Sección 2: API sin Autenticación (Autenticación 0)

1. Investiga y elige una API de acceso abierto.
2. Abre Postman y realiza una petición GET a la API.
3. Captura la evidencia con la respuesta JSON.



4. Responde:
 - a. ¿Por qué esta API no requiere autenticación? ¿Cuál es su rate limit?

R= Al ser una API publica y educativa su propósito es servir como una herramienta de aprendizaje y prototipado rápido, por lo que no maneja información sensible ni datos privados de usuarios reales. Aunque es abierta, JSONPlaceholder no especifica un límite estricto de peticiones por segundo en su página principal, pero generalmente las APIs gratuitas de este tipo aplican técnicas de Throttling (estrangulamiento) si detectan abusos (por ejemplo, miles de peticiones en un minuto) para evitar que sus servidores colapsen.

- b. ¿Cuáles serían los riesgos de no implementar autenticación?

R= Robo de información, Ataques de denegación de servicio, Alteración de datos y costos elevados.



Sección 3: Autenticación básica (Modo prueba)

Utiliza la API de [regres.in](https://reqres.in) para registrar un usuario. Esta es una API de prueba y no realiza un almacenamiento real de usuarios. Es útil para simular el flujo de registro y autenticación en aplicaciones.

La autenticación básica utiliza **Base64** para codificar las credenciales, pero no proporciona una capa de seguridad adicional, por lo que es fundamental utilizarla sobre HTTPS.

La codificación Base64 tiene como propósito principal convertir datos binarios (como texto o archivos) en una cadena de texto ASCII utilizando un conjunto de 64 caracteres (letras mayúsculas, minúsculas, números y dos símbolos: '+' y '/'). Esto asegura que los datos puedan ser transmitidos de manera segura a través de protocolos que no admiten datos binarios.

Por ejemplo:

```
const      texto      =      "eve.holt@reqres.in:pistol";
const      base64      =      Buffer.from(texto).toString('base64');
console.log(base64);           //      Salida:
ZXZlLmhvbHRAc mVxcmVzLmluOnBpc3RvbA==
```

Base64 no protege los datos, simplemente los transforma. Cualquiera que tenga acceso a la cadena codificada puede decodificar fácilmente:

```
const      base64      =      "ZXZlLmhvbHRAc mVxcmVzLmluOnBpc3RvbA==";
const      textoOriginal      =      Buffer.from(base64,
'base64').toString('utf-8');
console.log(textoOriginal); // Salida: eve.holt@reqres.in:pistol
```

Simula el registro de un usuario con el siguiente código:

1. Crea en la misma carpeta AA 2.2 API un archivo `basicaPost.js`
2. Transcribe los siguiente:





```
JS básicaPost.js > ...
1 import axios from "axios";
2 const registrarUsuario = async () => {
3   try {
4     const respuesta = await axios.post('https://reqres.in/api/register', {
5       email: 'eve.holt@reqres.in',
6       password: 'pistol'
7     });
8     console.log('Registro exitoso:', respuesta.data);
9   } catch (error) {
10    console.error('Error en el registro:', error.respuesta.data);
11  }
12 };
```

3. Observa los resultados en consola.

4. Crea otro archivo básicaGet.js y transcribe lo siguiente:

```
JS básicaGet.js > ...
1 import axios from "axios";
2 const obtenerUsuario = async () => {
3   try {
4     const response = await axios.get('https://reqres.in/api/users/4', {
5       headers: {
6         'Authorization': 'Basic ' + Buffer.from('eve.holt@reqres.in:pistol').toString('base64')
7       }
8     });
9     console.log('Datos del usuario:', response.data);
10  } catch (error) {
11    console.error('Error al obtener datos:', error.response.data);
12  }
13 };
14 obtenerUsuario();
```

El resultado muestra un objeto JSON que contiene información del usuario.

```
palomags@iMac-de-Paloma API % node básicaGet.js
Datos del usuario: {
  data: {
    id: 4,
    email: 'eve.holt@reqres.in',
    first_name: 'Eve',
    last_name: 'Holt',
    avatar: 'https://reqres.in/img/faces/4-image.jpg'
  },
  support: {
    url: 'https://contentcaddy.io?utm_source=reqres&utm_medium=json&utm_campaign=referral',
    text: 'Tired of writing endless social media content? Let Content Caddy generate it for you.'
  }
}
```

5. Modifica el código en el archivo básicaGet.js y usa credenciales incorrectas:

6. ¿Qué resultados arroja?



```
PS C:\Users\AdanA\Desktop\AA 2.2 API> node basicGet.js
Error al obtener datos: {
  error: 'missing_api_key',
  message: 'The x-api-key header is required for this endpoint.',
  hint: 'Create a free key at app.reqres.in and send it as x-api-key.',
  next_steps: [
    'Go to app.reqres.in/api-keys to get your key',
    'Add header: x-api-key: <your_key>',
    'For app user endpoints, use Authorization: Bearer <session_token> instead'
  ],
  docs_url: 'https://app.reqres.in/docs#authentication',
  example_curl: 'curl -H "x-api-key: YOUR_API_KEY" https://api.reqres.in/api/collections',
  _meta: {
    powered_by: 'ReqRes',
    docs_url: 'https://app.reqres.in/documentation',
    upgrade_url: 'https://app.reqres.in/upgrade',
    example_url: 'https://app.reqres.in/examples/notes-app',
    variant: 'v1_a',
    message: 'API key invalid or revoked. Create a new key in API Keys.',
    cta: { label: 'Get a key', url: 'https://app.reqres.in/api-keys' },
    context: 'invalid_key'
  }
}
```



2026
año de
**Margarita
Maza**



Sección 4: Uso de llaves en API

1. Investiga sobre el tipo de autenticación por llaves.
2. Desarrolla los pasos necesarios para poner a prueba la forma de realizar una petición a una API que maneja llaves.
3. Crea el archivo js, usando axios para crear la petición.
4. Puedes usar la API de OpenWeatherMap u otra similar.

```
clima.js > obtenerClima
1 import axios from 'axios';
2
3 const API_KEY = 'd53d8979e954a044c6f4cbc118d8c1b3';
4 const CIUDAD = 'Mexico';
5 const URL = `https://api.openweathermap.org/data/2.5/weather?q=${CIUDAD}&appid=${API_KEY}&units=metric`;
6
7 const obtenerClima = async () => {
8   try {
9     const respuesta = await axios.get(URL);
10    console.log(`--- Clima en ${respuesta.data.name} ---`);
11    console.log(`Temperatura: ${respuesta.data.main.temp}°C`);
12    console.log(`Humedad: ${respuesta.data.main.humidity}%`);
13    console.log(`Descripción: ${respuesta.data.weather[0].description}`);
14   } catch (error) {
15     console.error('Error al consultar el clima:', error.response ? error.response.data.message : error.message);
16   }
17 }
18
19 obtenerClima();
20
```

5. Responde:
 - a. ¿Cuál es la ventaja de usar una clave API?
 - b. ¿Qué sucede si compartes tu clave API con otros?

R= Control de acceso, identificación y seguridad

R= Existen agotamientos de cuota si la Api es gratuita se gastarían las peticiones, cargos monetarios, y revocación de cuenta.



Sección 5: Autenticación en Tokens

1. Investiga una API que use tokens.
2. Enviar usuario y contraseña en una petición POST de acuerdo con su documentación.

```
tokenPost.js > ...
1 import axios from 'axios';
2
3 const login = async () => {
4   try {
5     const respuesta = await axios.post('https://reqres.in/api/login', {
6       email: "eve.holt@reqres.in",
7       password: "cityslicka"
8     });
9     console.log('Token recibido:', respuesta.data.token);
10    return respuesta.data.token;
11  } catch (error) {
12    console.error('Error en login:', error.response.data);
13  }
14 };
15
16 login();
```

```
PS C:\Users\Adan\Desktop\AA 2.2 API> node tokenPost.js
Error en login: {
  error: 'missing_api_key',
  message: 'The x-api-key header is required for this endpoint.',
  hint: 'Send x-api-key for admin calls (/api/*) or Authorization: Bearer <token> for app-user calls (/app/*).',
  next_steps: [
    'Go to app.reqres.in/api-keys to get your key',
    'Add header: x-api-key: <your_key>',
    'For app user endpoints, use Authorization: Bearer <session_token> instead'
  ],
  docs_url: 'https://app.reqres.in/docs#authentication',
  example_curl: 'curl -H "x-api-key: YOUR_API_KEY" https://api.reqres.in/api/col
lections',
  _meta: {
    powered_by: 'ReqRes',
    docs_url: 'https://app.reqres.in/documentation',
    upgrade_url: 'https://app.reqres.in/upgrade',
    example_url: 'https://app.reqres.in/examples/notes-app',
    variant: 'v1_a',
    message: 'API key invalid or revoked. Create a new key in API Keys.',
    cta: { label: 'Get a key', url: 'https://app.reqres.in/api-keys' },
    context: 'invalid_key'
  }
}
```



3. Usa el token en el encabezado para acceder a los datos protegidos.

```
tokenGet.js > ...
1  import axios from 'axios';
2
3  const miToken = "QpWL5tke4Pnpja7X4";
4
5  const obtenerDatosProtegidos = async (token) => {
6    try {
7      const respuesta = await axios.get('https://reqres.in/api/users/2', {
8        headers: {
9          'Authorization': `Bearer ${token}`
10        }
11      });
12      console.log('Acceso concedido. Datos:', respuesta.data);
13    } catch (error) {
14      console.error('Error de acceso:', error.response.status, error.response.data);
15    }
16  };
17
18  obtenerDatosProtegidos(miToken);
```

```
PS C:\Users\Adan\Desktop\AA 2.2 API> node tokenGet.js
Error de acceso: 401 {
  error: 'missing_api_key',
  message: 'The x-api-key header is required for this endpoint.',
  hint: 'You sent a Bearer token but /api/* endpoints require x-api-key. Bearer tokens are for /app/* endpoints (app user access to scoped data).',
  next_steps: [
    'For admin operations: add x-api-key header instead of Bearer',
    'For app user data access: use /app/collections/:slug/records with your Bearer token',
    'Get your API key at app.reqres.in/api-keys'
  ],
  docs_url: 'https://app.reqres.in/docs#authentication',
  example_curl: 'curl -H "x-api-key: YOUR_API_KEY" https://api.reqres.in/api/collections',
  details: 'Authorization: Bearer was provided but this endpoint expects x-api-key header.',
  _meta: {
    powered_by: 'ReqRes',
    docs_url: 'https://app.reqres.in/documentation',
    upgrade_url: 'https://app.reqres.in/upgrade',
    example_url: 'https://app.reqres.in/examples/notes-app',
    variant: 'v1_a',
    message: 'API key invalid or revoked. Create a new key in API Keys.',
    cta: { label: 'Get a key', url: 'https://app.reqres.in/api-keys' },
    context: 'invalid_key'
  }
}
```



4. Prueba con un token inválido.

- ¿Por qué este método es más seguro que la autenticación básica?

R= Tiene exposición limitada la autenticación básica, el usuario y la contraseña se envían en cada petición. Con tokens, las credenciales solo se envían una vez para obtener el token, de igual forma tiene expiración, los tokens suelen tener un tiempo de vida corto. Si un atacante lo roba, solo tendrá acceso por un tiempo limitado, a diferencia de una contraseña que es permanente hasta que se cambie. Y por último en cuanto a la revocación, es más fácil invalidar un token específico desde el servidor sin obligar al usuario a cambiar su contraseña real.

- ¿Cuáles son las ventajas y desventajas de los tokens JWT?

R= como ventajas tiene stateless ósea que el servidor no necesita guardar sesiones en base de datos, también cuenta con una escalabilidad ideal para aplicaciones con muchos microservicios. Mientras que por otro lado sus desventajas son el tamaño y la seguridad ya que al contener datos de usuario en el encabezado http puede volverse pesado y si el token es robado, el atacante tiene acceso total hasta que el token expire.